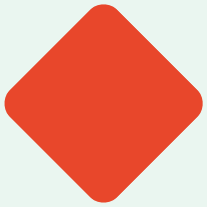
Git Basics:

What Every Beginner Must Know

1. What is Git?



- Git is a distributed version control system that tracks changes to your files.
- It lets you save different versions of your code, go back to previous versions, and work with others safely.
- Engineers use Git to manage source code, configuration files, documentation and infrastructure code.

2. Why Engineers Use Git?

- ✓ Track changes and history
- ✓ Work on features without breaking main code
- ✓ Collaborate with team members
- ✓ Revert to previous versions if something breaks
- ✓ Manage code safely in production environments

3. Git vs GitHub

Git	GitHub
• Version control system	• Online platform for hosting Git repositories
• Installed on your local machine	• Stores your code in the cloud
• Tracks changes to files	• Provides collaboration tools (pull requests, code reviews, issues)
• Works offline	• Not a replacement for Git
• Used with many remote platforms (GitHub, GitLab, Bitbucket, etc.)	• Uses Git under the hood

4. Key Git Terms

Term	Meaning
Repository (Repo)	A project folder that Git tracks. It contains your files and the complete history of changes.
Working Directory	Your local folder where you create and edit files.
Staging Area	A temporary area where you select changes to include in the next commit.
Commit	A saved snapshot of your changes with a unique ID and a message.

5. Basic Git Workflow

Working Directory
You create or edit files in your project folder.

git add →

Staging Area
You select the changes to include in the next commit.

git commit →

Local Repository
Your changes are saved as a commit in your local repo.

git push →

Remote Repository (GitHub)
You upload your commits to a remote repo to share with others.

6. Why Version Control Matters in DevOps?

- ✓ Keeps source and application code organized
- ✓ Helps engineers work together safely
- ✓ Provides history for debugging and auditing
- ✓ Supports CI/CD pipelines and automated deployments
- ✓ Reduces the risk of accidental changes in production
- ✓ Enables rollback if something goes wrong

☀ Key Takeaway

✓ Git helps you track changes, collaborate with others, and keep your code safe. GitHub is a popular platform for hosting Git repositories, but Git itself runs on your local machine.

Git Setup and Configuration Commands

1. Check Git Version

`>_` **git --version**

- Shows the installed Git version on your machine.
- Use this to confirm Git is installed and working.

Example Output:

```
git version 2.42.0
```

2. Set Your Name

`>_` **git config --global user.name "Your Name"**

- Sets your name for all repositories on this machine (global).
- Your name appears in commits so others know who made the changes.

Example:

```
git config --global user.name "Ann Felix"
```

3. Set Your Email

`>_` **git config --global user.email "you@example.com"**

- Sets your email for all repositories on this machine (global).
- Use the same email associated with your GitHub account.

Example:

```
git config --global user.email "ann@example.com"
```

4. View All Configuration

`>_` **git config --list**

- Displays all your Git configuration settings.
- Shows both global and local settings.
- Useful for checking if your name and email are set correctly.

Example Output (partial):

```
user.name=Ann Felix
user.email=ann@example.com
core.editor=vim
```

5. Show Configuration Source

`>_` **git config --show-origin --list**

- Shows where each configuration setting is coming from (global, local, or system).
- Helps you understand which file is setting each value.

Example Output (partial):

```
file:/home/ann/.gitconfig
user.name=Ann Felix
file:/home/ann/.gitconfig
user.email=ann@example.com
```

6. Local vs Global Configuration

Global Configuration

- Applies to all repositories on your machine.
- Set using **--global**.
- Stored in **~/.gitconfig**.
- Used for your default name, email and preferences.

Local Configuration

- Applies only to the current repository.
- Set without **--global**.
- Stored in **.git/config** inside the repository.
- Useful when you need a different name or email for a specific project (for example, work vs personal).

7. Useful Additional Commands

Command	What it does
git config user.name	Shows the current name for this repository.
git config user.email	Shows the current email for this repository.
git config --global --edit	Opens the global config file in your editor.
git config --edit	Opens the local config file in your editor (for the current repository).

☀ Important Notes

- ✓ Use the same name and email as your GitHub account for consistency.
- ✓ You can use different names/emails for different projects using local configuration.
- ✓ If you work on personal and work projects, keep them separate to avoid confusion.

Beginner Mistakes

- ✗ Forgetting to set name and email.
- ✗ Using a different email from your GitHub account.
- ✗ Thinking global settings apply to only one project.
- ✗ Not checking the configuration when commits show the wrong name or email.

Quick Check

- ✓ Run **git --version** and confirm Git is installed.
- ✓ Set your name and email using **--global**.
- ✓ Run **git config --list** to verify the settings.
- ✓ Use **git config --show-origin --list** to see where settings come from.

Creating and Cloning Git Repositories

1. Create a New Local Repository

`>_` **git init**

- Creates a new Git repository in the current directory.
- Initializes a hidden .git folder to track changes.
- You can now start adding and committing your files.

Example:

```
mkdir my-project
cd my-project
git init
```

→ Initialized empty Git repository in /home/ann/my-project/.git/

2. Clone an Existing Repository

`>_` **git clone <repo-url>**

- Creates a copy of a remote repository on your local machine.
- Downloads the entire project history.
- You automatically get a local copy and a remote named origin.

Example:

```
git clone https://github.com/veriqta/demo.git
```

3. Clone a Specific Branch

`>_` **git clone --branch <branch-name> <repo-url>**

- Clones only a specific branch instead of the default branch (usually main).
- Useful when you want to work on a feature branch or older version.

Example:

```
git clone --branch feature/login https://github.com/veriqta/demo.git
```

4. View Remote Repositories

`>_` **git remote -v**

- Shows the remote repositories linked to your local repository.
- Displays the remote name (usually origin) and the URLs for fetch and push.

Example Output:

```
origin  https://github.com/veriqta/demo.git (fetch)
origin  https://github.com/veriqta/demo.git (push)
```

5. Local vs Remote Repositories

Local Repository

- Lives on your computer.
- You control the files.
- Works offline.
- Stored in a .git folder.
- Used for development, testing and local changes.

git push →

← git pull

Remote Repository

- Lives on a server (e.g. GitHub, GitLab).
- Accessible over the internet.
- Used for collaboration.
- Stores the complete project history.
- Usually named origin.

6. Simple Workflow

Create or Clone → **Make Changes** → **Add & Commit** → **Push to Remote**

7. Practice: Create a Repository

1. Create a new project folder.
2. Initialize a Git repository.
3. Create a file and add some content.
4. Add and commit the file.
5. Check the repository status.
6. (Optional) Create a GitHub repo and push your code.

☀ Key Takeaway

- ✓ Use **git init** when starting a new project.
- ✓ Use **git clone** when working on an existing project.
- ✓ Local repos are for development, remote repos are for collaboration and backup.
- ✓ Every cloned repo already has a remote named origin.

Checking Git

Repository Status

1. Check Repository Status

`>_` **git status**

- Shows the current state of your working directory and staging area.
- Tells you which files are modified, staged, untracked, or deleted.
- Very useful before committing or pushing changes.

2. Short Status Output

`>_` **git status -sb**

- Shows a shorter, simplified status.
- Great for quick checks.
- Displays your current branch and a compact list of changed files.

Example Output:

```
## main
 M  app.py
?? notes.txt
```

M = modified
?? = untracked

3. View Changes in Files

`>_` **git diff**

- Shows the exact changes in files that are modified but not yet staged.
- Helps you review your changes before adding them.

Example Output (partial):

```
diff --git a/app.py b/app.py
index e69de29..4b825dc 100644
--- a/app.py
+++ b/app.py
@@ -0,0 +1,2 @@
+print("Hello World")
```

4. View Summary of Changes

`>_` **git diff --stat**

- Shows a summary of changes in each file (modified lines).
- Useful for a quick overview.

Example Output:

```
app.py    | 2 ++
readme.md | 5 ++++ -
2 files changed, 6 insertions(+), 1 deletion(-)
```

5. Understanding File States

State	Meaning	How it appears	Example
Tracked	Git is already tracking the file.	No special label (if unchanged)	app.py
Modified	Tracked file has changes.	M (in status)	app.py
Staged	Changes added to staging area.	A or M (in staged area)	app.py
Untracked	New file not yet tracked by Git.	?? (in status)	notes.txt

6. Understanding git status Output

```
On branch main
Changes to be committed:
  (use "git restore --staged ..." to unstage)
        modified:   app.py
        new file:   script.sh

Changes not staged for commit:
  (use "git add ..." to update what will be
committed)
        modified:   readme.md

Untracked files:
  (use "git add ..." to include in what will be
committed)
        notes.txt
```

☀ Key Takeaway

- ✓ Use **git status** to see what is going on before you commit.
- ✓ M means modified.
- ✓ A means added.
- ✓ ?? means untracked.
- ✓ Use **git diff** to review changes.
- ✓ Use **git diff --stat** for a summary.

8. Common Beginner Mistakes

- ✗ Committing without checking git status.
- ✗ Forgetting to add new files.
- ✗ Ignoring modified files.
- ✗ Not understanding the difference between staged and modified files.
- ✗ Thinking untracked files are automatically committed.

Adding and Committing Changes

1. Add a Specific File

>_ **git add file.txt**

- Adds a specific file to the staging area.
- The file is now ready to be committed.
- Only this file will be staged.

Example:

```
git add file.txt
```

2. Add All Changes

>_ **git add .**

- Adds all new, modified and deleted files in the current directory to the staging area.
- Use this when you want to stage everything.

Example:

```
git add .
```

3. Commit the Staged Changes

>_ **git commit -m "message"**

- Saves the staged changes to the local repository with a commit message.
- Each commit has a unique ID.
- Good commit messages make your history easy to understand.

Example:

```
git commit -m "Add user login feature"
```

4. View Staged Changes

>_ **git diff --cached**

- Shows the exact changes that are in the staging area (what you have added).
- Useful to review your changes before committing.

Example Output:

```
diff --git a/app.py b/app.py
index e69de29..4b825dc 100644
--- a/app.py
+++ b/app.py
+print("Hello World")
```

5. Staging vs Committing

Working Directory
Your files and changes (not yet tracked for commit).

git add →

Staging Area
You select the changes to include in the next commit.

git commit →

Local Repository
The staged changes are saved as a commit with a unique ID and message.

Staging

- Selects specific changes.
- Lets you review what to commit.
- You can stage part of a file.
- Nothing is saved permanently yet.

Committing

- Saves the staged changes.
- Creates a commit with a message.
- Adds to your project history.
- You can push commits to a remote repo.

6. Common Beginner Mistakes

- ✕ Forgetting to stage files before committing.
- ✕ Using **git add .** when you only wanted one file.
- ✕ Committing without a meaningful message.
- ✕ Staging sensitive files like .env or credentials.
- ✕ Not checking the staged changes with **git diff --cached**.
- ✕ Thinking **git add** automatically commits.
- ✕ Accidentally staging large or unnecessary files (logs, build files, etc.).

7. Practice: Add and Commit a File

1. Create a file.
2. Check the status.
3. Add the file.
4. Commit with a message.
5. Verify the commit.

Example:

```
echo "Hello Git" > hello.txt
git status
git add hello.txt
git commit -m "Add hello.txt"
git log --oneline
```

☀ 8. Key Takeaway

- ✓ Use **git add** to stage changes.
- ✓ Use **git commit** to save staged changes.
- ✓ Always review with **git diff --cached**.
- ✓ Write clear and meaningful commit messages.
- ✓ Only stage files you actually want to commit.
- ✓ This gives you control and keeps your repository history clean.

Viewing Git Commit History

1. View Commit History

`>_` **git log**

- Shows the complete commit history for the current branch.
- Displays commit ID, author, date and commit message.
- Press q to exit.

Example Output:

```
commit 3a5f9e2c7d1e4b8a
Author: Ann Felix
Date:   Mon Oct 2 10:15:30 2026
+0000

    Add login feature
```

2. Compact Commit History

`>_` **git log --oneline**

- Shows a shortened, single-line view of each commit.
- Useful for a quick overview.
- Shows commit ID (short) and commit message.

Example Output:

```
3a5f9e2  Add login feature
7b1c4d8  Fix bug in dashboard
9c2e1a3  Update README
4d8f9c0  Initial commit
```

3. Visual Commit History

`>_` **git log --graph --oneline --all**

- Shows a visual (graph) view of all branches.
- Displays branch structure and commit history.
- Very useful when working with multiple branches.

Example Output:

```
* 3a5f9e2 (HEAD -> main) Add login
* 7b1c4d8 Fix bug in dashboard
| * 2c9d1e7 (feature/ui) Update UI
|/
* 9c2e1a3 Update README
* 4d8f9c0 Initial commit
```

4. View Details of a Specific Commit

`>_` **git show <commit-id>**

- Shows detailed information about a specific commit.
- Displays the commit message and exact changes made.

Example Output (partial):

```
commit 3a5f9e2c7d1e4b8a
Author: Ann Felix
Date:   Mon Oct 2 10:15:30 2026
+0000

    Add login feature
diff --git a/app.py b/app.py
+   def login():
+       return "Login page"
```

5. View the Latest Commit (HEAD)

`>_` **git show HEAD**

- Shows details of the most recent commit on your current branch.
- HEAD always points to the latest commit.

Example Output (partial):

```
commit 3a5f9e2c7d1e4b8a
Author: Ann Felix
Date:   Mon Oct 2 10:15:30 2026
+0000

    Add login feature
diff --git a/app.py b/app.py
+   def login():
+       return "Login page"
```

6. Understanding Commit IDs and HEAD

Commit ID (SHA)

- A unique identifier for each commit.
- Usually 40 characters (e.g. 3a5f9e2c7d...).
- You can use the first few characters (e.g. **3a5f9e2**) if unique.
- Identifies a specific snapshot of your code.

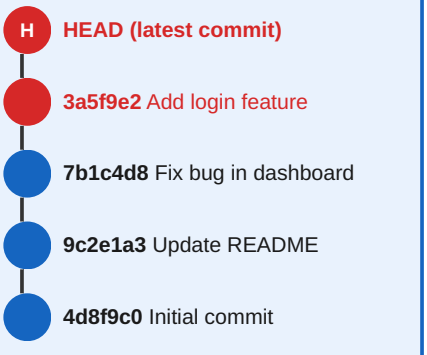
HEAD

- A pointer to the latest commit on your current branch.
- HEAD moves forward when you make a new commit.
- You can also use HEAD~1, HEAD~2, etc. to refer to previous commits.

7. Common Options with git log

Command	What It Does
`git log`	Shows full commit history.
`git log --oneline`	Shows a compact one-line view.
`git log --graph --oneline --all`	Shows a visual graph of all branches.
`git log -n 5`	Shows the last 5 commits.
`git log --author="Ann"`	Shows commits by a specific author.
`git log --since="2026-10-01"`	Shows commits since a specific date.

8. Simple Commit History Diagram



9. Common Beginner Mistakes

- ✕ Not understanding what a commit ID is.
- ✕ Using an incomplete commit ID that matches multiple commits.
- ✕ Thinking HEAD is a branch (HEAD is a pointer).
- ✕ Not using **git log** to review history before making changes.
- ✕ Confusing the current branch with other branches when viewing logs.

☀ 10. Key Takeaway

- ✓ Use **git log** to understand your history.
- ✓ Use **--oneline** for a quick view.
- ✓ Use **--graph** to see branch structure.
- ✓ Each commit has a unique ID (SHA).
- ✓ HEAD always points to the latest commit on your current branch.

Git Branch Commands Explained

1. View Local Branches

`>_` **git branch**

- Lists all local branches in your repository.
- The currently active branch is marked with *.

Example Output:

```
* main
  feature/login
  bugfix/ui
```

2. View All Branches

`>_` **git branch -a**

- Lists all local and remote branches.
- Local branches are shown first.
- Remote branches are prefixed with origin/.

Example Output:

```
* main
  feature/login
  remotes/origin/main
  remotes/origin/feature/login
```

3. Create and Switch to a New Branch

`>_` **git switch -c feature/login**

- Creates a new branch and switches to it immediately.
- Useful for working on a new feature, bug fix, or experiment.

Example Output:

```
Switched to a new branch 'feature/login'
```

4. Switch to an Existing Branch

`>_` **git switch main**

- Switches to an existing branch (e.g. main).
- Your working directory will now match the files in that branch.

Example Output:

```
Switched to branch 'main'
```

5. Delete a Branch

`>_` **git branch -d feature/login**

- Deletes a local branch after it has been merged.
- Use -D (uppercase) to force delete if it hasn't been merged.

Example Output:

```
Deleted branch feature/login (was 7b1c4d8).
```

6. Why Use Feature Branches?

- ✓ Keep the main branch stable and production-ready.
- ✓ Work on new features in isolation.
- ✓ Allow multiple people to work on different features at the same time.
- ✓ Make it easier to review code before merging.
- ✓ Reduce the risk of breaking the main branch.
- ✓ Support better collaboration and version control in DevOps environments.

7. Common Branch Workflow

Create branch `git switch -c feature/login` → **Make changes** (edit files) → **Commit changes** `git add .` `git commit -m "message"` → **Open a pull request** (on GitHub/GitLab) → **Merge to main** (after review)

8. Branch Types (Common Examples)

Branch Type	Example Name
Feature branch	feature/login
Bug fix branch	bugfix/navbar
Hotfix branch	hotfix/security-patch
Release branch	release/v1.0.0
Experiment branch	experiment/new-ui

9. Useful Additional Branch Commands

Command	What It Does
git branch -r	List only remote branches.
git branch -m	Rename a branch.
git branch --merged	List branches that have been merged.
git branch --no-merged	List branches not yet merged.
git switch -	Switch to the previous branch.

☀ 10. Key Takeaway

- ✓ Use feature branches to keep your main branch stable.
- ✓ Create a new branch for each feature or fix.
- ✓ Make your changes, commit, and open a pull request for review.
- ✓ Delete branches after they are merged.
- ✓ Use clear and descriptive branch names.
- ✓ Branching makes Git safer, cleaner, and easier to work with in teams.

Git Merge Commands and Workflows

1. Merge a Branch

`>_` **git merge <branch>**

- Merges the changes from the specified branch into your current branch.
- You are usually on the target branch (e.g. main) before merging.
- If there are no conflicts, Git automatically combines the changes.

Example:

```
# Switch to main first
git switch main
# Merge feature branch
git merge feature/login
```

2. Merge with a Merge Commit

`>_` **git merge --no-ff <branch>**

- Always creates a merge commit even if a fast-forward merge is possible.
- Keeps the branch history visible in the commit history.
- Useful for tracking feature branches.

Example:

```
git merge --no-ff featur/login
# Creates a merge commit with
# a clear history
```

3. Abort a Merge

`>_` **git merge --abort**

- Stops the merge process if you are in the middle of a merge and want to cancel it.
- Returns your repository to the state before the merge started.
- Useful when conflicts are too complex or you want to try a different approach.

Example:

```
# Abort an ongoing merge
git merge --abort
```

4. Fast-Forward vs Merge Commit

Fast-Forward Merge

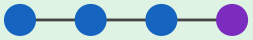
- Happens when there are no diverging changes.
- The target branch simply moves forward to the latest commit.
- No extra merge commit is created.

Before:



main feature

After (Fast-Forward):



main

Merge Commit

- Happens when there are diverging changes.
- Creates a new merge commit that combines both histories.
- Keeps the full branch history visible.

Before:



main feature

After (Merge Commit):



main

5. Simple Feature Branch Merge Diagram

1. **Create a feature branch**
   git switch -c feature/login
2. **Make commits on the feature branch**
   (feature/login)
3. **Switch to main branch**
   git switch main
4. **Merge the feature branch**
   git merge feature/login
5. **Result: main now has the changes from feature/login**

6. How Merge Conflicts Arise

- A merge conflict happens when Git cannot automatically combine changes from both branches.
- This usually occurs when the same lines in a file are modified differently in each branch.
- You must manually resolve the conflict before completing the merge.

Example Conflict (app.py)

```
<<<<<<< HEAD
print("Login page")
=======
print("Login page
UI")
>>>>>>> feature/
login
```

What the Markers Mean?

`<<<<<<< HEAD`	Your current branch changes (e.g. main)
`=======`	Separator between the conflicting changes
`>>>>>>> feature/login`	Changes from the branch you are merging (e.g. feature/login)

7. Resolve a Merge Conflict

1. Open the conflicting file.
2. Look for the conflict markers (<<<<<<<, =======, >>>>>>>).
3. Edit the file and keep the correct changes.
4. Remove the conflict markers.
5. Save the file.
6. Add the file to staging. **git add <file>**
7. Complete the merge. **git commit**

☀ 8. Key Takeaways

- ✓ Use **git merge** to combine branches.
- ✓ Use **--no-ff** to keep branch history.
- ✓ Use **--abort** to cancel a merge.
- ✓ Fast-forward merges don't create extra commits.
- ✓ Merge commits keep the full history.
- ✓ Conflicts happen when the same lines are changed in both branches.
- ✓ Manually resolve conflicts, then add and commit.
- ✓ Merging is a key part of teamwork in DevOps and software development.

Working with Remote Repositories

1. View Remote Repositories

`>_` **git remote -v**

- Lists the remote repositories connected to your local repository.
- Shows the short name (e.g. origin) and the URL for fetch and push.

Example Output:

```
origin  https://github.com/veriqta/demo.git (fetch)
origin  https://github.com/veriqta/demo.git (push)
```

2. Add a Remote Repository

`>_` **git remote add origin URL**

- Connects your local repository to a remote repository (usually named origin).
- You only need to do this once when setting up a new repository.

Example:

```
git remote add origin \
https://github.com/veriqta/demo.git
```

3. Fetch from Remote Repository

`>_` **git fetch origin**

- Downloads the latest changes from the remote repository.
- Updates your local remote-tracking branches (e.g. origin/main).
- Does not automatically merge the changes into your working branch.

Example Output:

```
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
From https://github.com/veriqta/demo
 * [new branch]      main     -> origin/main
```

4. Push to Remote Repository

`>_` **git push -u origin main**

- Uploads your local commits to the remote repository.
- The -u flag sets the upstream branch, so future pushes can be done with just **git push**.

Example Output:

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (5/5), done.
To https://github.com/veriqta/demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

5. Change Remote Repository URL

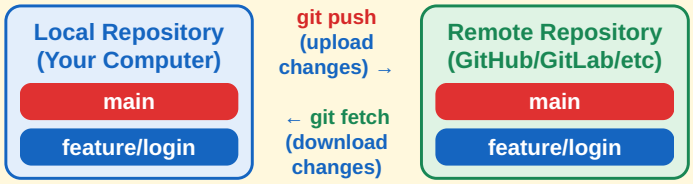
`>_` **git remote set-url origin URL**

- Updates the remote URL if it has changed (e.g. switching from HTTPS to SSH).

Example:

```
git remote set-url origin \
https://github.com/veriqta/new-demo.git
```

6. How Local and Remote Branches Connect



- Local branches track remote branches (e.g. main ↔ origin/main).
- Pushing sends your local commits to the remote branch.
- Fetching downloads changes from the remote branch.
- Pulling (**git pull**) fetches and merges the changes into your current branch.

7. Useful Related Commands

Command	What It Does
git remote -v	View remotes and URLs.
git remote add origin URL	Add a remote repository.
git fetch origin	Download latest changes.
git push -u origin main	Push and set upstream branch.
git remote set-url origin URL	Update remote URL.
git branch -r	List remote branches.
git checkout -b feature origin/feature	Create a local branch from a remote.

☀ 8. Key Takeaway

- ✓ Use **git remote -v** to check your remotes.
- ✓ Use **git remote add origin URL** to connect to a remote repo.
- ✓ Use **git fetch origin** to get the latest changes.
- ✓ Use **git push -u origin main** to upload your commits.
- ✓ Use **git remote set-url origin URL** to update the remote URL.
- ✓ Local and remote branches are linked and keep your code in sync across environments.

Git Pull, Push, and Fetch Explained

1. Fetch from Remote

`>_` **git fetch**

- Downloads the latest changes from the remote repository to your local machine.
- Does not merge the changes into your working branch.
- Updates your remote-tracking branches (e.g. origin/main).

Example Output:

```
remote: Enumerating objects: 5,
done.
remote: Counting objects: 100%
(5/5), done.
From https://github.com/veriqta/
demo
 * [new branch]      main -> origin/
main
```

2. Pull from Remote

`>_` **git pull**

- Fetches the latest changes from the remote repository and merges them into your current branch.
- Updates your local branch with remote changes.
- Can create a merge commit if there are divergent changes.

Example Output:

```
From https://github.com/veriqta/
demo
 * branch            main ->
FETCH_HEAD
Updating 1a2b3c4..5d6e7f8
Fast-forward
 file.txt | 2 ++
 1 file changed, 2 insertions(+)
```

3. Pull with Fast-Forward Only

`>_` **git pull --ff-only**

- Only updates if a fast-forward merge is possible.
- Does not create a merge commit.
- Fails if your local branch has diverged from the remote.
- Useful to avoid accidental merge commits.

Example Output (success):

```
Updating 1a2b3c4..5d6e7f8
Fast-forward
 file.txt | 2 ++
 1 file changed, 2 insertions(+)
```

Example Output (failed):

```
fatal: Not possible to fast-
forward, aborting.
```

4. Push to Remote

`>_` **git push**

- Uploads your local commits to the remote repository.
- Pushes the current branch (only if it is tracking a remote branch).
- Use -u on the first push to set the upstream branch.

Example Output:

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (5/5), done.
To https://github.com/veriqta/
demo.git
   1a2b3c4..5d6e7f8  main -> main
```

5. Push a Specific Branch

`>_` **git push origin feature/login**

- Pushes your local feature/login branch to the remote repository named origin.
- Useful when working on feature branches.
- Creates the remote branch if it doesn't exist.

Example Output:

```
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Writing objects: 100% (8/8), done.
To https://github.com/veriqta/
demo.git
 * [new branch]      feature/login ->
feature/login
```

6. Fetch vs Pull vs Push

Fetch

- Downloads changes only.
- Updates remote-tracking branches.
- Does NOT merge into your code.

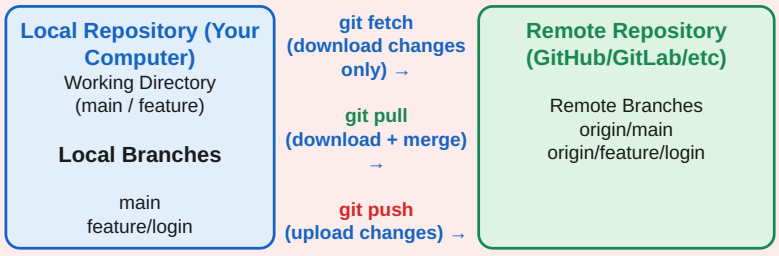
Pull

- Downloads and merges changes.
- Updates your current branch.
- Can create a merge commit.

Push

- Uploads your local commits.
- Sends changes to the remote repository.
- Usually requires write access.

7. How It Works (Simple Diagram)



8. Avoiding Accidental Integration

1. Use **git fetch** first to see what changed on the remote before merging.
2. Review changes with **git log**, **git diff**, or **git show**.
3. Use **git pull --ff-only** if you only want fast-forward merges.
4. Avoid pulling into feature branches unless intended.
5. Keep your local branch updated regularly.
6. Communicate with your team to avoid overwriting others' work.

☀ 9. Key Takeaway

- ✓ Use **git fetch** to get remote changes.
- ✓ Use **git pull** to fetch and merge.
- ✓ Use **git pull --ff-only** to avoid unwanted merge commits.
- ✓ Use **git push** to upload your commits.
- ✓ Use **git push origin feature/login** to push a specific branch.
- ✓ Always review remote changes before pulling.

Undoing Git Changes Safely

1. Restore a Modified File

`>_` **git restore file.txt**

- Discards changes in your working directory for the specified file.
- Restores the file to the last committed version.
- Useful when you don't want to keep local changes.

Example:

```
git restore file.txt
```

2. Unstage a File

`>_` **git restore --staged file.txt**

- Removes a file from the staging area.
- Keeps your changes in the working directory.
- Useful when you added a file by mistake.

Example:

```
git restore --staged file.txt
```

3. Revert a Commit

`>_` **git revert COMMIT**

- Creates a new commit that undoes the changes from a specific commit.
- Does not delete history.
- Safe to use on shared repositories.

Example:

```
git revert a1b2c3d
```

4. Reset (Soft)

`>_` **git reset --soft HEAD~1**

- Moves the branch pointer back by one commit.
- Keeps the changes in the staging area.
- Useful when you want to redo the commit message or split the commit.

Example:

```
git reset --soft HEAD~1
```

5. Reset (Hard)

`>_` **git reset --hard HEAD~1**

- Moves the branch pointer back by one commit.
- Discards ALL changes in the working directory and staging area.
- Use with caution (you cannot easily recover the changes).

Example:

```
git reset --hard HEAD~1
```

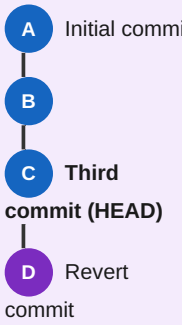
6. Which Commands Can Discard Work?

Command	Can Discard Work?
git restore file.txt	Yes (discard local changes in working directory)
git restore --staged file.txt	No (only unstages, keeps changes)
git revert COMMIT	No (creates a new commit)
git reset --soft HEAD~1	No (keeps changes staged)
git reset --hard HEAD~1	Yes (discard commits and all changes)

7. How the Commands Affect Your Changes

Command	Working Directory	Staging Area (Index)	Commit History	Use Case
git restore file.txt	Reverts file	No change	No change	Discard local file changes
git restore --staged file.txt	No change	Removes from staging	No change	Unstage a file
git revert COMMIT	No change	No change	Adds new commit that undoes changes	Safely undo a commit (on shared repos)
git reset --soft HEAD~1	No change	Keeps changes staged	Moves branch back	Redo or edit last commit
git reset --hard HEAD~1	Discarded	Discarded	Moves branch back	Completely remove last commit and changes

8. Visual Comparison



git reset --soft HEAD~1
Moves back (keeps changes staged)

git reset --hard HEAD~1
Moves back (discard changes)

git revert C
Creates new commit that undoes C

☀ 9. Key Takeaways

- ✓ Use **git restore** to discard unwanted changes in your working directory.
- ✓ Use **git restore --staged** to unstage files without losing changes.
- ✓ Use **git revert** to safely undo a commit (does not rewrite history).
- ✓ Use **git reset --soft** to move back a commit but keep your changes.
- ✓ Use **git reset --hard** only when you are sure you want to permanently discard changes.
- ✓ Be careful with commands that discard work, especially on shared repositories.

Git Stash Commands

Temporarily Saving Unfinished Work

1. Stash Your Changes

git stash push

- Temporarily saves your uncommitted changes.
- Cleans your working directory so you can switch branches or work on something else.
- Your changes are stored in a stash (not lost).

Example:

```
git stash push -m "WIP: login page"
```

2. View Stash List

git stash list

- Shows all your saved stashes in order (newest first).
- Each stash has an index (stash@{0}, stash@{1}, etc.).
- Shows the message you added when stashing.

Example:

```
git stash list
stash@{0}: On main: WIP: login page
stash@{1}: On feature: API changes
stash@{2}: On main: update README
```

3. Apply the Latest Stash (and Remove)

git stash pop

- Applies the most recent stash to your working directory.
- Restores your changes and removes the stash from the list.
- If there are conflicts, you will need to resolve them.

Example:

```
git stash pop
On branch main
Changes popped from stash@{0}.
```

4. Apply a Stash (Keep It)

git stash apply

- Applies a specific stash to your working directory.
- Keeps the stash in the list (does not remove it).
- Useful when you want to test the changes but keep a backup.

Example:

```
git stash apply stash@{0}
```

5. Delete a Stash

git stash drop

- Removes a specific stash from the list.
- Useful when you no longer need the saved changes.

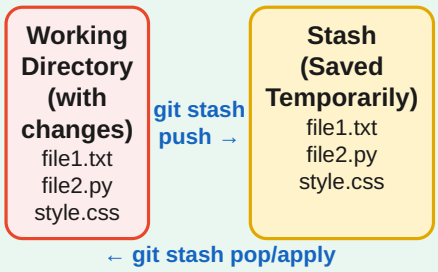
Example:

```
git stash drop stash@{0}
```

Delete all stashes:

```
git stash clear
```

6. How Git Stash Works (Flow)



7. Common Stash Use Cases

Scenario	Command
Save unfinished work to switch branches	**git stash push -m "message"**
List saved stashes	**git stash list**
Apply the latest stash and remove it	**git stash pop**
Apply a specific stash (but keep it)	**git stash apply stash@{n}**
Delete a specific stash	**git stash drop stash@{n}**
Delete all stashes	**git stash clear**

8. Example Workflow

1. You are working on a feature but need to switch to main.

```
git stash push -m "WIP: feature X"
```

2. Switch to main and work on something else.

```
git switch main
```

3. When ready, restore your changes.

```
git stash pop
```

☀ 9. Key Takeaways

- ✓ Use **git stash** to temporarily save unfinished work.
- ✓ Use **git stash list** to view all stashes.
- ✓ Use **git stash pop** to apply and remove the latest stash.
- ✓ Use **git stash apply** to apply without removing.
- ✓ Use **git stash drop** to delete a specific stash.
- ✓ Stashes are local to your machine.
- ✓ Stash is useful when switching branches, fixing urgent issues, or keeping work safe.

Resolving Git Merge Conflicts

1. What is a Merge Conflict?

- A merge conflict happens when Git cannot automatically merge changes because the same part of a file has been modified in different branches.
- Git stops the merge and asks you to resolve the conflict manually.
- Conflicts are common in team environments and are a normal part of version control.

2. Understanding Conflict Markers

- When a conflict occurs, Git adds special markers in the file to show the conflicting changes.
- You will see three sections:
  - Your current branch changes (HEAD)
  - The incoming branch changes
  - The separator lines

3. Example Conflict Markers in a File

```
1  <<<<<<< HEAD
2  console.log("Hello from main");
3  =======
4  console.log("Hello from feature");
5  >>>>>>> feature/login
```

<<<<<<< HEAD	Your current branch changes
=======	Separator (not part of final code)
>>>>>>> feature/login	Incoming branch changes

4. Check the Status

>_ **git status**

- Shows which files have conflicts.
- Files with conflicts are marked as unmerged.
- Use this to see what needs your attention.

Example Output:

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")

Unmerged paths:
  (use "git add ..." to mark resolution)
        both modified:   app.py
```

5. Check for Other Issues

>_ **git diff --check**

- Checks for whitespace and other potential issues in files.
- Useful after resolving conflicts to ensure the file is clean.

Example Output:

```
app.py:3: trailing whitespace.
app.py:5: no newline at end of file.
```

6. Resolve the Conflict

1. Open the conflicted file.
2. Review the changes from both sides.
3. Edit the file to keep the correct content.
4. Remove the conflict markers <<<<<<< ======= >>>>>>>.
5. Save the file.

Example (resolved file):

```
1  console.log("Hello from main");
2  console.log("Hello from feature");
```

7. Stage the Resolved File

>_ **git add <file>**

- After fixing the conflict, stage the file to mark it as resolved.
- You must stage all resolved files before continuing the merge.

Example:

```
git add app.py
```

8. Continue the Merge

>_ **git merge --continue**

- Tells Git that you have resolved the conflicts.
- Git will complete the merge and create a merge commit.

Example Output:

```
[main 7f9a2c1] Merge branch 'feature/login'
 1 file changed, 2 insertions(+).
```

9. Practical Example Workflow

1. Try to merge a branch (conflict occurs).

```
git merge feature/login
```

2. Check the status.

```
git status
```

3. Open the conflicted file and resolve it.
4. Stage the file.

```
git add app.py
```

5. Continue the merge.

```
git merge --continue
```

10. Simple Conflict Example (Full Flow)

main branch (app.py)

```
1 console.log("Hello");
2 console.log("Main branch");
```

feature/login branch (app.py)

```
1 console.log("Hello");
2 console.log("Feature branch");
```

→

After merge (resolved)

```
1 console.log("Hello");
2 console.log("Main branch");
3 console.log("Feature branch");
```

☀ 11. Key Takeaways

- ✓ A merge conflict means Git cannot automatically merge your changes.
- ✓ Use **git status** to see conflicted files.
- ✓ Understand and remove conflict markers.
- ✓ Use **git diff --check** to find other issues.
- ✓ Use **git add** to stage resolved files.
- ✓ Use **git merge --continue** to complete the merge.
- ✓ Conflicts are normal and part of teamwork.

Git Rebase Commands for Beginners

1. What is Git Rebase?

- Rebase moves your commits to a new base (e.g. main) and rewrites the commit history.
- It replays your commits one by one on top of the latest changes.
- It creates a cleaner, linear history without merge commits.

Key Point:

```
Rebase does not create a merge commit.
It rewrites history.
```

2. Rebase a Branch onto Main

`>_` **git rebase main**

- Moves your current branch commits to the latest main.
- Applies your commits on top of main one by one.
- Useful to keep a clean commit history.
- Run this while on your feature branch.

Example:

```
# switch to your feature branch
git switch feature/login
# rebase onto main
git rebase main
```

3. Continue a Rebase

`>_` **git rebase --continue**

- Used after fixing conflicts during a rebase.
- Stages the resolved files and continues applying the remaining commits.
- Repeat until the rebase completes.

Example (after resolving conflicts):

```
git add
git rebase --continue
```

4. Abort a Rebase

`>_` **git rebase --abort**

- Stops the rebase process.
- Returns your branch to the state it was in before the rebase started.
- Useful when you want to cancel the rebase and keep your original history.

Example:

```
git rebase --abort
```

5. Merge vs Rebase

Merge (creates a merge commit)

- Keeps the original history.
- Preserves the exact commit structure.
- Shows where branches were merged.
- Good for shared branches.



main feature: D, E → Merge commit (created)

Rebase (linear history)

- Rewrites commit history.
- Moves your commits on top of the latest main.
- Creates a cleaner, linear history.
- No merge commit.



Commits D and E moved on top of main (new commit IDs)

6. Simple Commit History Diagram

Before Rebase



main



feature — Feature branch is behind main. It has its own commits (D, E).

After Rebase



feature (rebased) — Feature branch commits are replayed on top of the latest main.

7. Why Rebasing Shared History Can Cause Problems

⚠

- Rebasing rewrites commit IDs.
- If you rebase a branch that others have already pulled, their history will be different.
- This can cause confusion, failed pulls, and duplicate commits.
- Avoid rebasing shared branches like main or branches used by your team.
- Use rebase mainly on your local feature branches before opening a pull request.

8. Practical Workflow (Rebase)

1. Switch to your feature branch.

```
git switch feature/login
```

2. Get the latest changes.

```
git fetch origin
git rebase main
```

3. Resolve any conflicts (if they occur).

```
git add
```

4. Continue the rebase.

```
git rebase --continue
```

9. When to Use Merge vs Rebase

Situation	Use Merge	Use Rebase
Personal feature branch (before PR)	✗	✓
Shared branch (main, develop, team branch)	✓	✗
You want a clean, linear history	✗	✓
You want to preserve the exact history	✓	✗

☀ 10. Key Takeaways

- ✓ Use **git rebase main** to replay your commits on the latest main.
- ✓ Use **git rebase --continue** after resolving conflicts.
- ✓ Use **git rebase --abort** to cancel a rebase.
- ✓ Rebase creates a cleaner, linear history without merge commits.
- ✓ Avoid rebasing shared branches.
- ✓ Use rebase on your local feature branches to keep history clean.

Recovering Lost Work with Git Reflog

1. What is Git Reflog?

- Git reflog keeps a log of where your HEAD and branch references have been, even after resets, rebases and other operations.
- It records changes to your local repository for a limited time.
- Useful for recovering commits that seem "lost".

Key Point:

```
Reflog is a safety net. It
tracks your repository's recent
history.
```

2. View the Reflog

>_ **git reflog**

- Shows a list of recent changes to your repository (HEAD history).
- Each entry has a reference like HEAD@{0}, HEAD@{1}, etc.
- Shows the commit ID, operation description and time.

Example Output:

```
a1b2c3d (HEAD -> main) HEAD@{0}:
reset: moving to HEAD~1
f4e5d6c HEAD@{1}: commit: add
log
9a8b7c6 HEAD@{2}: checkout:
moving
from feature/login to main
3c2d1e0 HEAD@{3}: commit: update
README
```

3. View Reflog for a Specific Reference

>_ **git reflog show HEAD**

- Shows the reflog for HEAD only (most common).
- You can also view other references, for example:

git reflog show main

git reflog show feature/login

Example:

```
git reflog show HEAD
f4e5d6c HEAD@{0}: reset: moving
to HEAD~1
a1b2c3d HEAD@{1}: commit: add
feature
9a8b7c6 HEAD@{2}: checkout:
moving
from feature/login to main
```

4. Inspect a Lost Commit

>_ **git show COMMIT**

- Use the commit ID from the reflog to see the details of the lost commit.
- Check the files, changes and commit message to confirm it's the one you want.

Example:

```
git show f4e5d6c
commit f4e5d6c...
Author: Your Name
Date:   ...

    Add login feature
diff --git a/app.py b/app.py
+ print("Login page")
```

5. Recover the Commit (Create a Branch)

>_ **git branch recovery COMMIT**

- Creates a new branch pointing to the lost commit so you can recover the work.
- You can then switch to the branch and continue working.

Example:

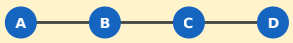
```
git branch recovery f4e5d6c
git switch recovery
# Now your recovered commit
# is available on the recovery
branch.
```

6. Common Recovery Scenario

1. You accidentally ran **git reset --hard HEAD~1** and lost your commit.
2. Use **git reflog** to find the commit ID.
3. Inspect it with **git show COMMIT**.
4. Create a recovery branch **git branch recovery COMMIT**.
5. Switch to the branch and verify your files are back.

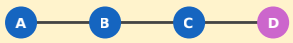
7. Visual Example: Recovering a Lost Commit

Before Reset (commits exist)



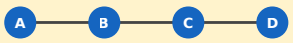
main — HEAD (latest commit)

After git reset --hard HEAD~1



HEAD (moved back) — Commit D (seems lost)

Recover using reflog



main — recovery: Recovered commit on new branch

8. Reflog Limits and Retention

⚠

- Reflog is local to your local repository (not shared with others).
- Entries are kept for a limited time (default is usually 90 days, can vary by Git version/config).
- Once expired and garbage collected, the reflog entries and commits may no longer be available.
- Reflog is not a backup solution, but it is very useful for recent mistakes.

☀ 9. Key Takeaways

- ✓ Use **git reflog** to find lost commits.
- ✓ Use **git show COMMIT** to verify the commit.
- ✓ Use **git branch recovery COMMIT** to recover it.
- ✓ Reflog helps after accidental resets, rebases and other history changes.
- ✓ Reflog is local and has a limited retention period.
- ✓ Always confirm the commit before creating a branch.

Git Tags and Release Versioning

1. What are Git Tags?

- Tags are permanent references to specific commits.
- They are commonly used to mark release versions (e.g. v1.0.0).
- Unlike branches, tags do not move when new commits are added.
- Tags help you identify and reproduce exact versions of your code.

Use Case:

- Marking software releases
- Tracking stable versions
- Deploying a specific version

2. List Tags

>_ **git tag**

- Shows all tags in your repository.
- Tags are usually named with version numbers like v1.0.0, v1.1.0, etc.

Example:

```
$ git tag
v1.0.0
v1.1.0
v2.0.0
```

3. Create an Annotated Tag

>_ **git tag -a v1.0.0 -m "Release v1.0.0"**

- Creates an annotated tag with a message.
- Annotated tags include metadata such as the tagger name, email, date and message.
- This is the recommended way to create release tags.

Example:

```
$ git tag -a v1.0.0 -m "Release
v1.0.0"
```

4. View Tags with Description

>_ **git tag -n**

- Shows a list of tags with their annotation messages.
- Helps you quickly see what each tag is for.
- Only works for annotated tags.

Example:

```
$ git tag -n
v1.0.0   Release v1.0.0
v1.1.0   Bug fixes and improvements
v2.0.0   Major new features
```

5. Push a Tag to Remote

>_ **git push origin v1.0.0**

- Pushes a specific tag to the remote repository.
- Tags are not pushed automatically when you push your branches.
- You can also push all tags using **git push --tags**.

Example:

```
$ git push origin v1.0.0
# or push all tags
$ git push --tags
```

6. View a Tagged Commit

>_ **git show v1.0.0**

- Shows the commit details for the tag.
- Displays the tag message, commit information and the changes in that release.

Example:

```
$ git show v1.0.0
tag v1.0.0
Tagger: Your Name
Date:   Fri Oct 10 10:00:00 2026
+0000
Release v1.0.0

commit a1b2c3d4e5f6...
Author: Your Name
Date:   Fri Oct 10 09:58:00 2026
+0000
    Initial release
```

7. Lightweight vs Annotated Tags

Feature	Lightweight Tag	Annotated Tag
Created with	**git tag v1.0.0**	**git tag -a v1.0.0 -m "msg"**
Includes message	No	Yes
Includes metadata	No	Yes
Best for	Temporary tags	Official release versions
Example	**v1.0.0**	**v1.0.0** (with message)

8. How Tags Support Software Releases

- Tags mark stable, production-ready versions of your software.
- They allow you to easily go back to a specific version if needed.
- Used in CI/CD pipelines to trigger releases.
- Help teams and users identify which version is deployed.
- Commonly follow semantic versioning (e.g. v1.0.0, v1.1.0, v2.0.0).

Typical Release Flow

1. Develop and test code
2. Create a release tag (e.g. v1.0.0)
3. Push the tag to remote
4. Deploy the tagged version to production

9. Simple Release Workflow

v0.9.0	v1.0.0	v1.1.0	v2.0.0
Initial release	First stable version	Bug fixes	Major new features

☀ 10. Key Takeaways

- ✓ Use **git tag** to list tags.
- ✓ Use **git tag -a v1.0.0 -m "message"** for release tags.
- ✓ Use **git tag -n** to see tag descriptions.
- ✓ Use **git push origin v1.0.0** to push a specific tag.
- ✓ Use **git show v1.0.0** to view tag details.
- ✓ Tags help you manage and deploy software releases.
- ✓ Always use annotated tags for official releases.

Git Ignore and Repository Security

1. What is .gitignore?

- .gitignore is a file that tells Git which files and folders to ignore.
- Ignored files are not tracked and will not be included in commits.
- Useful for logs, temporary files, local configuration, build artifacts and sensitive files.
- It keeps your repository clean and focused on important code.

2. Create a .gitignore File

- Create a file named .gitignore in the root of your repository.
- Add patterns for files and folders you want to ignore.
- Use comments (#) to explain rules.

Example .gitignore:

```
# Logs
*.log
logs/

# Temporary
files
*.tmp
.cache/

# IDE files
.vscode/
.idea/
```

```
# OS files
.DS_Store
Thumbs.db

# Environment
# and local
config
.env
config/
local.json
```

3. Check Why a File is Ignored

`>_` **git check-ignore -v**

- Shows which rule is causing a file to be ignored.
- Useful when you are not sure why a file is not showing in git status.

Example:

```
$ git check-ignore -v logs/app.log
.gitignore:2:*.log    logs/app.log
```

- This shows that the file is ignored because of the ***.log** rule on line 2 in .gitignore.

4. Remove a Tracked File (Keep it Locally)

`>_` **git rm --cached <file>**

- If a file was already committed and you now want to ignore it, remove it from tracking but keep a local copy.
- Then add it to .gitignore.

Example:

```
$ git rm --cached config/local.json
$ echo "config/local.json"
>> .gitignore
$ git commit -m "Stop tracking local
config"
```

5. Common Files to Ignore

Type	Examples
Logs	*.log, logs/
Temporary files	*.tmp, *.cache, tmp/
Build artifacts	dist/, build/, target/
IDE files	.vscode/, .idea/
OS files	.DS_Store, Thumbs.db
Local config	.env, config/local.json
Dependency folders	node_modules/, .venv/
Test reports	coverage/, test-results/
System files	*.swp, *.bak, *~

6. .gitignore Does NOT Protect Already Committed Secrets

- .gitignore only stops new files from being tracked.
- If a secret (e.g. API key, password) was already committed, it will still exist in the repository history.
- You must remove the file from the repository and rotate the secret if it was exposed.

⚠ Warning

Never commit secrets to Git. Even if you add them to .gitignore later, they can still be recovered from the commit history.

7. Protecting Credentials in DevOps Repositories

- Never store real credentials in your repository.
- Use environment variables for sensitive values.
- Use a .env file locally and add it to .gitignore.
- Use secure secret management tools in production (e.g. AWS Secrets Manager, Azure Key Vault, GitHub Secrets).
- Regularly rotate credentials if accidentally exposed.

Example .env (ignored):

```
DB_PASSWORD=your_password_here
API_KEY=your_api_key_here
```

Better Approach (Environment Variables)

```
# In your application
import os

api_key = os.getenv("API_KEY")
db_password = os.getenv("DB_PASSWORD")
```

8. Practical Example

1. Create a .gitignore file.
2. Add rules to ignore logs, temp files and local config.
3. Remove any already tracked sensitive files.

```
git rm --cached .env
```

4. Commit the changes.

```
git commit -m "Update .gitignore"
```

9. Directory Structure Example

```
my-project/
|-- .gitignore      ← Ignore rules here
|-- app/
|-- logs/           → Ignored (logs/)
|-- config/         → Ignored (config/local.json)
|-- .env            → Ignored (.env)
`-- README.md       → Tracked
```

☀ 10. Key Takeaways

- ✓ Use .gitignore to keep your repository clean.
- ✓ Use **git check-ignore -v** to understand ignored files.
- ✓ Use **git rm --cached** to remove already tracked files.
- ✓ Ignore logs, temporary files, build artifacts and local configuration.
- ✓ .gitignore does not protect secrets already committed.
- ✓ Use environment variables and secret management tools.
- ✓ Never commit real credentials to your repository.

Git Troubleshooting Cheat Sheet

1. fatal: not a git repository

fatal: not a git repository (or any of the parent directories): .git

Why it happens?

- You are not inside a Git repository.
- The .git folder is missing.
- You may be in the wrong directory.

How to fix?

```
# Check your current directory
pwd
ls -la

# Initialize a new repository (if needed)
git init
```

2. fatal: refusing to merge unrelated histories

fatal: refusing to merge unrelated histories

Why it happens?

- You are trying to merge or pull from a repository with a completely different history (e.g. existing project and a new repo).

How to fix?

```
# If you are sure you want to merge
git pull origin main --allow-unrelated-histories

# Or when merging a branch
git merge  --allow-unrelated-histories
```

3. error: failed to push some refs

error: failed to push some refs to 'https://github.com/user/repo.git'

Why it happens?

- Remote has new changes you don't have.
- Your local branch is behind the remote.
- You don't have the correct permissions.

How to fix?

```
# Pull the latest changes first
git pull origin main

# Then push again
git push origin main

# If using a feature branch
git push -u origin feature/branch-name
```

4. Permission denied (publickey)

Permission denied (publickey). fatal: Could not read from remote repository.

Why it happens?

- Your SSH key is not added to GitHub.
- You are using the wrong SSH key.
- Your SSH agent is not running.

How to fix?

```
# Check if your SSH key is loaded
ssh -T git@github.com

# Start ssh-agent and add your key
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_rsa

# Make sure your public key is added
# to your GitHub account
```

5. CONFLICT (content)

CONFLICT (content): Merge conflict in file.txt. Automatic merge failed; fix conflicts and then commit the result.

Why it happens?

- The same part of a file was changed in different branches.
- Git cannot automatically decide which change to keep.

How to fix?

```
# Check which files have conflicts
git status
# Open the file and look for conflict markers
<<<<<<< HEAD
=======
>>>>>>> branch-name
# Edit the file, keep the correct content
# Then stage and commit
git add file.txt
git commit
```

6. Other Common Issues

fatal: could not read Username for 'https://github.com': No such device or address

- You are not authenticated. Use a personal access token (PAT) or SSH key.

fatal: remote origin already exists

- A remote named origin already exists. Use **git remote -v** to check and update it if needed.

error: src refspec main does not match any

- The branch does not exist locally. Check your branch name with **git branch**.

✓ Everything up-to-date

- There are no new changes to push or pull. Your local and remote are in sync.

7. Useful Diagnostic Commands

Command	What it does
git status	Shows the current repository status.
git branch	Lists your local branches.
git branch -r	Lists remote branches.
git remote -v	Shows the remote repository URLs.
git log --oneline	Shows commit history.
git config --list	Shows your Git configuration.
ssh -T git@github.com	Tests your SSH connection (GitHub).
git fsck	Checks the integrity of your repository.

8. Safe First Steps When You See an Error

1. Read the full error message carefully.
2. Understand what triggered the error.
3. Check your current directory (pwd).
4. Run **git status** to see the state.
5. Check your remotes (**git remote -v**).
6. Look at recent changes (**git log --oneline**).
7. Search the exact error message if unsure.

☀ 9. Key Takeaways

- ✓ Always read and understand the full error message.
- ✓ Use diagnostic commands before making changes.
- ✓ Keep your local and remote repositories in sync.
- ✓ Configure SSH keys or PATs correctly.
- ✓ Resolve conflicts carefully.
- ✓ Avoid force pushing on shared branches.
- ✓ When in doubt, make a backup branch before trying a fix.

Git in Real DevOps and CI/CD Workflows

1. Developer Commits Code

Make code changes in your local repository.

```
$ git status
$ git add .
$ git commit -m "Add new feature"
```

What happens?

- Your changes are saved locally as a commit with a unique ID.
- Commits track who, what, when and why.

2. Push Triggers CI/CD Pipeline

Local Repo git push → **Remote Repository (GitHub)**

What happens?

- You push your commits to a remote branch.
- The push event triggers an automated CI/CD pipeline (GitHub Actions, Jenkins, etc).
- The pipeline is defined in a configuration file (e.g. .github/workflows/).

3. Automated Tests and Security Checks Run

Build — Install dependencies and build the application.

Unit Tests — Run automated tests to verify functionality.

Security Checks — Scan for vulnerabilities (Dependencies, SAST, Container scans, etc.).

Quality Checks — Linting, code quality and policy checks.

Goal:
Ensure the code is safe, stable and meets quality standards before it can be merged.

4. Pull Request Review and Merge

Open a Pull Request (PR) from your feature branch to main.

↓

Team Review — Code is reviewed by team members.

↓

Comments & Changes — Feedback is provided and changes are made if needed.

↓

Merge — After approval, the PR is merged into the main branch.

5. Build and Deployment

Code Merged to main → **Build** Build the application (e.g. compile, create Docker image). → **Package** Package artifacts (e.g. Docker image, JAR, etc.). → **Deploy** Deploy to staging environment. → **Run Tests** Integration tests and smoke tests. → **Deploy to Production** Deploy to production after approval (manual or automatic).

6. Git's Role in Popular DevOps Tools

GitHub Actions

- Uses your Git repository events (push, PR, tag).
- Defined in .github/workflows/.
- Automates build, test, security scan and deployment.

Example trigger:

```
on:
  push:
    branches:
[ main ]
```

Jenkins

- Pulls code from your Git repository.
- Runs build, test and deployment stages.
- Uses webhooks or polling to detect changes.
- Supports complex pipelines and integrations.

Example (Jenkinsfile):

```
stage('Checkout') {
  steps {
    git 'https://
github.com/..'
  }
}
```

GitOps (ArgoCD / Flux)

- Monitors a Git repository for changes in configuration (e.g. Kubernetes manifests).
- Automatically syncs changes to your cluster.
- Keeps infrastructure and applications in the desired state.

Example flow:

```
Git repo (manifests)
  |
ArgoCD/Flux
  |
Deploy to Kubernetes
```

7. End-to-End Example Workflow

1. Developer commits and pushes code.
2. GitHub receives the push and triggers the pipeline.
3. CI pipeline builds the code and runs tests and security checks.
4. Team reviews pull request and merges to main.
5. CD pipeline builds and deploys the application.
6. Application is deployed to staging/production.
7. Monitoring and feedback ensures a healthy release.

8. Key Commands in Workflow

Command	Purpose
git add .	Stage changes.
git commit -m "msg"	Create a commit.
git push origin branch	Push to remote repository.
git pull origin main	Update your local repository.
git checkout -b feature/login	Create a new feature branch.
git merge main	Merge changes.
git tag v1.0.0	Tag a release version.

9. Benefits

- ✓ Automates testing and deployment.
- ✓ Reduces manual errors.
- ✓ Identifies issues early.
- ✓ Improves code quality and security.
- ✓ Enables faster and more reliable releases.
- ✓ Supports collaboration and review.
- ✓ Forms the foundation for modern DevOps and GitOps workflows.

☀ 10. Key Takeaways

- ✓ Git is the source of truth for your code and configuration.
- ✓ A push can trigger automated CI/CD pipelines.
- ✓ Always use pull requests for review.
- ✓ Automated tests and security checks improve reliability.
- ✓ Git integrates with tools like GitHub Actions, Jenkins and GitOps to enable continuous delivery.